Hadamard product in deep learning: Introduction, Advances and Challenges

Grigorios G Chrysos, Yongtao Wu, Razvan Pascanu, Philip Torr, Volkan Cevher

arXiv:2504.13112v1 [cs.LG] 17 Apr 2025

Abstract—While convolution and self-attention mechanisms have dominated architectural design in deep learning, this survey examines a fundamental yet understudied primitive: the *Hadamard product*. Despite its widespread implementation across various applications, the Hadamard product has not been systematically analyzed as a core architectural primitive. We present the first comprehensive taxonomy of its applications in deep learning, identifying four principal domains: higher-order correlation, multimodal data fusion, dynamic representation modulation, and efficient pairwise operations. The Hadamard product's ability to model nonlinear interactions with linear computational complexity makes it particularly valuable for resource-constrained deployments and edge computing scenarios. We demonstrate its natural applicability in multimodal fusion tasks, such as visual question answering, and its effectiveness in representation masking for applications including image inpainting and pruning. This systematic review not only consolidates existing knowledge about the Hadamard product's role in deep learning architectures but also establishes a foundation for future architectural innovations. Our analysis reveals the Hadamard product as a versatile primitive that offers compelling trade-offs between computational efficiency and representational power, positioning it as a crucial component in the deep learning toolkit.

Index Terms—representation learning, deep learning, Hadamard product, high-order correlations, multimodal fusion, gating mechanism, masking.

1 Introduction

Deep neural networks (DNNs) have demonstrated unparalleled success in learning complicated concepts. Diverse tasks (such as image classification [1], image generation [2], [3], image inpainting [4], [5], language understanding [6], [7], sequence modeling [8], [9], sequence transduction [10] and multimodal learning [11], [12]) are nowadays predominantly modelled with DNNs. One of the pillars behind the success of DNNs (and the focus of this work) is the choice of the architectural components, and more specifically the type of correlations captured.

Two of the most popular architectural components used across domains and tasks are the convolutional layer and the self-attention [10]. A convolutional layer represents an element as a weighted sum of the neighboring input elements. Concretely, given a datapoint $\boldsymbol{Z}$ (e.g., a sequence of text tokens or an image), a convolutional layer expresses each output as $\sum_{i,j} w_{i,j} z_{i,j}$ for (learnable) weights $\boldsymbol{W}$. A convolutional layer captures linear correlations between the input elements, which limits the contextual information. On the contrary, the self-attention (SA) represents nonlinear interactions through a matrix multiplication of the input elements, i.e., products of the form $(\boldsymbol{W}_1\boldsymbol{Z}) \cdot (\boldsymbol{W}_2\boldsymbol{Z})$ for (learnable) weights $\boldsymbol{W}_1, \boldsymbol{W}_2$. However, SA suffers from a quadratic computational cost over the number of input elements [13], which results in a significant computational overhead for deep networks. Instead, the Hadamard product captures *nonlinear interactions with a linear computational cost*. The Hadamard product is a pairwise operator that accepts two input tensors and outputs their element-by-element product in each position. Representing nonlinear interactions with a linear computational cost is critical in the era of deep networks, e.g., in order to use them on edge devices [14], [15] or especially for large language models and foundational models [16] that are computationally intensive.

Capturing nonlinear interactions is especially important when we have different modalities. As humans, we perceive the world through various sensory input, e.g., visual and auditory modalities, that offer complementary information, while they share a common underlying structure. Enabling neural networks to process and understand multimodal information is critical for important applications, such as medical analysis. Clearly, data from different modalities might vary widely in volume, making processing more difficult. A core part of processing multimodal data is the fusion of those volumes. A way to fuse information is through naive concatenation, however this expresses only linear interactions [38]. Richer, nonlinear interactions can be captured with tensor decompositions, such as the popular CP decomposition. If we use the CP decomposition, the Hadamard product emerges naturally through standard properties (cf. the mixed product [39] or Lemma 2). Naturally, the Hadamard product has been widely applied to multimodal fusion tasks, such as in visual question answering [11] or multimodal medical analysis [40].

Beyond the linear computational cost and the multimodal fusion, we argue that the Hadamard product lies at the heart of deep learning (DL) with far-reaching applications. In this work, our aim is to contextualize those

- *GC is with the University of Wisconsin-Madison.*
- *YW, VC are with the Department of Electrical Engineering, Ecole Polytechnique Federale de Lausanne (EPFL), Switzerland.*
- *RP is with DeepMind.*
- *PT is with the Department of Engineering Science, University of Oxford. Corresponding author's e-mail: chrysos@wisc.edu.*

TABLE 1: Recent works using the Hadamard product. Beyond the seminal works that Hadamard product has been used, the Hadamard product exists in new applications, e.g., in recent language models or parameter-efficient fine-tuning.

Category	Task	Method	Publication
High-order interactions	Video action recognition	[17]	ICCV'23
	Image classification	MONet [18]	ICLR'24
	Masked language modeling	M2 [19]	NeurIPS'23
	Classification/segmentation	RMT [20]	CVPR'24
Multimodal fusion	Video Question Answering	[21]	CVPR'24
	Image generation	MultiDiffusion [22]	ICML'23
	Image generation	DenseDiffusion [23]	ICCV'23
Adaptive modulation	Language modeling	xLSTM [8]	NeurIPS'24
	Classification/segmentation	V-RWKV [24]	ICLR'25
	Pruning	PX [25]	CVPR'24
	3D texture generation	[26]	ICCV'23
	Conditional image generation	LayoutFlow [27]	ECCV'24
	Language modeling	[28]	NeurIPS'24
	Language modeling	Mamba [29]	COLM'24
	Language modeling	Mamba2 [30]	ICML'24
	Language modeling	HGRN2 [31]	COLM'24
	Language modeling	GLA [32]	ICML'24
Efficient operator	Image classification	Poly-SA [33]	PAMI'23
	Image classification	StarNet [34]	CVPR'24
	Masked language modeling	FLASH [15]	ICML'22
	Parameter-efficient fine-tuning	LoHa [35]	ICLR'22
	Parameter-efficient fine-tuning	HiRA [36]	ICLR'25
	Parameter-efficient fine-tuning	MLAE [37]	Arxiv, May'24

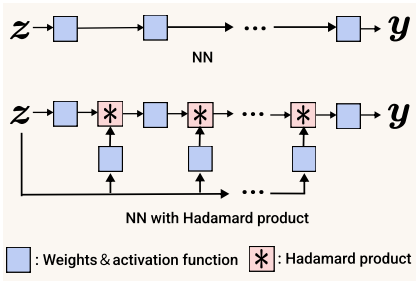
applications of the Hadamard product and to frame it as a core operator in deep learning. Concretely, our analysis focuses on four categories where the Hadamard product has been instrumental. The first category revolves around representing high-order correlations, e.g., as in StyleGAN [2] or SENet [1]. The second category develops the fusion of multimodal data, where different communities have independently used the Hadamard product as a fusion scheme. Then, we present the use of Hadamard product for dynamic filtering of features, e.g., in case there are corrupted parts of the image in masking [4], [5], and dynamic modulation of the weights, e.g., in Dropout to avoid overfitting or in LSTM to avoid vanishing/exploding gradients [8]. Masking is also utilized in language modeling, e.g., with linear attention models [41], to enable efficient training on text sequences. This is critical for more computationally efficient training of models predicting the next output token. The last category we revisit the computational efficiency of the Hadamard product, e.g., for reducing the complexity of standard self-attention blocks. By linking these applications of the Hadamard product and presenting them in a unified perspective, we hope to provide a more comprehensive understanding of this important operator in deep learning. This could also help to connect seemingly unrelated components, such as the inductive bias of polynomial networks [42] and the inductive bias of LSTM. In Table 1 we review some recent works in each category, with additional, significant works being mentioned in the respective sections, e.g., Tables 2 and 3. All in all, the contributions of this survey are the following:

- This work presents the first taxonomy of the Hadamard product, establishing significant connections among different categories, such as multimodal fusion and high-order interactions.
- Within each category, we explore how various concepts influence the application of the Hadamard product, for example, masking in adaptive modulation.
- We highlight the key theoretical properties of the Hadamard product that have been developed in isolation. We discuss the impact of these properties, such as the effect of spectral bias on high-order interactions.
- We identify important open problems in Section 7, which are expected to stimulate future research on this topic.

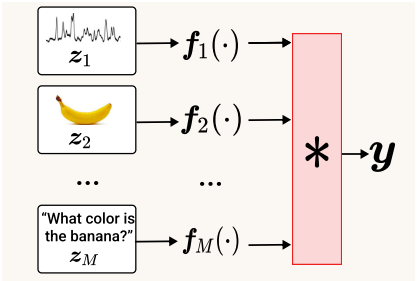
Brief history: The term Hadamard product was originally termed in [43]. The influential book of [44] argues that the most popular references attribute the first use of this product in the manuscripts of Jacques Hadamard [45], [46]. However, more recent books [47] argue that this element-wise product had actually emerged at least few years earlier in Moutard's work. The works of Hadamard and Schur [48] are among the first to (re-)invent the element-wise product. A by-product of the numerous re-inventions of this product is that there is no universally accepted symbol for the Hadamard product [47]. In this work, we will interchangeably use the term Hadamard and element-wise product, while we denote this product with the symbol '$*$'.

Relationship to other surveys: The rise in deep learning components and tasks has led to numerous surveys, covering areas like attention networks [49], recurrent networks [50], Dropout [51], tensors [39], [52], [53], and multimodal learning [54]. While the Hadamard product is related to some of these topics, such as Dropout and multimodal fusion, it is not the primary focus of these surveys. Our work fills this gap by explicitly addressing the Hadamard product and its wide range of applications, complementing existing surveys.

Paper outline: We cover the first application of the Hadamard product in deep learning in Section 2, while we provide a verbose taxonomy in Fig. S2. Concretely, the Hadamard product is used there for capturing high-order correlations among the input elements, e.g., expressing a

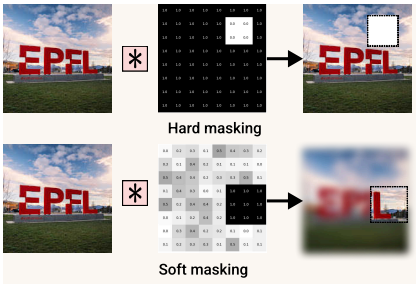


(a) **High-order interactions:** Regular NN (top) and how to include higher-order interactions (bottom)

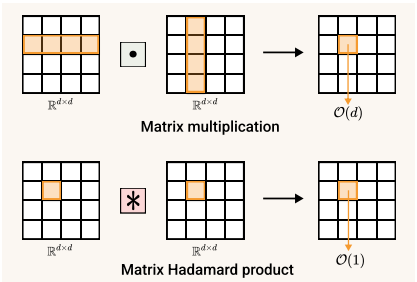


(b) **Multimodal Fusion**

(c) **Adaptive modulation:** Casual language modeling with linear attention



(d) **Adaptive modulation:** Hard masking (top) and soft masking (bottom)



(e) **Efficient operator**

(f) **Adaptive modulation and efficient operator:** Weight pruning

Fig. 1: Six core areas where the Hadamard product has been widely used in deep learning era. (a) High-order correlations between the input elements are captured. Those correlations can augment the linear interactions of the typical layers, e.g., dense or convolutional layers. (b) As humans we generally perceive the world through different senses, which often offer complementary information. Similarly, machine learning (ML) models can extract complementary information from different sources and then meld them together to make an informed decision. (c) During the pre-training of language modeling, we mask attention to the key of future tokens for each query so that the model does not use information from the next token when predicting the next token. (d) Hard masking and soft masking via Hadamard product for the image in the input space. (e) The Hadamard product has recently been used as an alternative operator to the matrix multiplication, e.g., in order to accelerate the popular Self-Attention. (f) Weight pruning can be viewed as applying a Hadamard product to the original weights, effectively zeroing out certain parameters. Among those core areas, we identify four parent categories and links between them (e.g. weight pruning). To our knowledge, this taxonomy is novel and allows us to establish concrete connections between seemingly disparate works within the same category, such as masking for inpainting and causal language modeling. To facilitate further research, we have also compiled the diverse open-source links in Table S4.

synthesized image as high-order correlations between the elements of the noise vector. This is a topic that has become relevant the last few years following the success of StyleGAN and other similar works. Another major area of research in the Hadamard product is the feature fusion, especially in the context of multimodal learning. We cover various feature fusion schemes using the Hadamard product in Section 3. A third area of development of networks with the Hadamard product is in masking, e.g., as in inpainting or Dropout. The idea is to modulate the input/features, such that the network can (exclusively) focus on certain areas of the representation. This idea has also been explored in the form of gating, which selects dynamically the critical features. Both masking and the gating mechanisms are detailed in Section 4. Furthermore, the Hadamard product has been featured recently as a more (computationally) efficient operator over the more costly matrix multiplication (e.g. in the context of self-attention) as covered in Section 5. Six popular applications of the four parent categories are visually depicted in Fig. 1. Beyond the technical and computational aspects covered, the Hadamard product results in a number of interesting theoretical properties, which are studied in Section 6. Lastly, we conduct a thorough discussion on the benefits of the Hadamard product, as well as highlight existing limitations and identify interesting avenues for future research on the topic in Section 7.

Notation: Matrices (vectors) are denoted by uppercase (lowercase) boldface letters e.g., $\boldsymbol{X}$, ($\boldsymbol{x}$). Tensors are denoted with boldface, calligraphic letters, e.g., $\boldsymbol{\mathcal{X}}$. Detailed notation, along with core mathematical properties appear in Section S.1. In this work, the input to the network is denoted with (vector) $\boldsymbol{z}$ or (matrix) $\boldsymbol{Z}$.

Definition 1.1 (Hadamard product). The *Hadamard product* between two tensors $\boldsymbol{\mathcal{X}}, \boldsymbol{\mathcal{Y}}$ is denoted as $(\boldsymbol{\mathcal{X}} * \boldsymbol{\mathcal{Y}}) \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_M}$ and is defined as

$$(\boldsymbol{\mathcal{X}} * \boldsymbol{\mathcal{Y}})_{i_1, i_2, \ldots, i_M} \doteq (\boldsymbol{\mathcal{X}})_{i_1, i_2, \ldots, i_M} \cdot (\boldsymbol{\mathcal{Y}})_{i_1, i_2, \ldots, i_M} \ .$$

Notice that the Hadamard product between vectors and

matrices are special cases of this definition. Although the Hadamard product requires that the two tensors share the same dimensions, each I_i can be different from I_j for $i \neq j$. Further information on tensors, tensor decompositions and products can be found in the classic review papers of [39], [52], while [53] provides a recent overview of tensor in deep learning.

2 High-order interactions

Beyond the linear interactions used in neural networks, polynomial expansions have been explored for approximating signals with unknown analytic expressions. The Stone-Weierstrass theorem [55] guarantees that any smooth function can be approximated by a polynomial expansion, with its multivariate extension covered in standard textbooks, e.g., in [56] (pg 19). In this section, we analyze how such a polynomial expansion can be used to augment the linear interactions of the typical layers used in DNNs. Let $\boldsymbol{z} \in \mathbb{R}^d$ denote the input, e.g., an image patch, and let $\boldsymbol{y} \in \mathbb{R}^o$ be the target, e.g., a class label. A polynomial expansion of the input $\boldsymbol{z}$ can be considered for approximating the target $\boldsymbol{y}$. That is, a vector-valued function $f(\boldsymbol{z}) : \mathbb{R}^d \rightarrow \mathbb{R}^o$ expresses the multivariate polynomial expansion of degree N:

$$\boldsymbol{y} = f(\boldsymbol{z}) = \sum_{n=1}^{N} \left(\boldsymbol{\mathcal{W}}^{[n]} \prod_{j=2}^{n+1} \times_j \boldsymbol{z} \right) + \boldsymbol{\beta} \,, \tag{1}$$

where $\boldsymbol{\beta} \in \mathbb{R}^o$ and $\{\boldsymbol{\mathcal{W}}^{[n]} \in \mathbb{R}^{o \times \prod_{m=1}^{n} \times_m d}\}_{n=1}^{N}$ are the learnable parameters. The form of Eq. (1) can approximate smooth functions (for large N). The number of parameters required to accommodate high-order interactions of the input increases exponentially with the desired degree of the polynomial which is impractical for high-dimensional data. Therefore, a form of parameter reduction is applied to each work to make polynomial expansions practical.

Group Method of Data Handling (GMDH) [57] was one of the first works to learn quadratic polynomials. To reduce the parameteres, GMDH defines a predetermined set of interactions between the input elements. The method was later extended in high-degree polynomials [58]. Contrary to the aforementioned predefined interactions, quadratic [59], [60] and high-degree expansions [61], [62] have been learned in a data-driven fashion. In particular, pi-sigma networks [63] use a single multiplicative unit over the input elements. That is, given an input $\boldsymbol{z} \in \mathbb{R}^d$ the output $\boldsymbol{y} \in \mathbb{R}^o$ in pi-sigma networks can be expressed as $\boldsymbol{y} = \sigma(*_{j=1}^{J} \boldsymbol{h}_j)$, with σ an element-wise activation function and J the order of the interactions. The vector $\boldsymbol{h}_j$ represents the j^{th} column-vector of the hidden representation $\boldsymbol{H} = \boldsymbol{\mathcal{W}} \times_3 \boldsymbol{z} + \boldsymbol{B}$. The parameters $\boldsymbol{\mathcal{W}} \in \mathbb{R}^{o \times J \times d}$, $\boldsymbol{B} \in \mathbb{R}^{o \times J}$ are learnable.

More recently, there are a number of works that use polynomial expansions in ML. Various quadratic polynomials of the form $f_1(\boldsymbol{z}) + f_2(\boldsymbol{z}) * f_3(\boldsymbol{z})$ for appropriate functions of f_1, f_2, f_3 are used in [6], [18], [20], [64], [65], [66], [67], [68], [69], [70], [71], [72]. For instance, in the Gated Linear Unit (GLU) [73], they assume $f_1(\boldsymbol{z}) = \boldsymbol{0}, f_2(\boldsymbol{z}) = \boldsymbol{W}\boldsymbol{z}$ and $f_3(\boldsymbol{z}) = \sigma(\boldsymbol{V}\boldsymbol{z})$ for learnt weights $\boldsymbol{W}, \boldsymbol{V}$. In Highway networks [64] the functions $f_1(\boldsymbol{z}) = \boldsymbol{z}, f_2(\boldsymbol{z}) = \boldsymbol{z} - H(\boldsymbol{z})$ and $f_3(\boldsymbol{z}) = \sigma(\boldsymbol{W}\boldsymbol{z})$ are used.

Beyond second-order interactions, polynomial expansions of arbitrary degree have been introduced, which propose various ways to reduce the parameters of Eq. (1). A straightforward solution is to assume a low-rank structure in each of the tensors. The CP decomposition along with Lemma 2 is used to obtain the output $\boldsymbol{y}$ as an N^{th} degree expansion [74]:

$$\boldsymbol{y} = \boldsymbol{\beta} + \boldsymbol{C}_{1,[1]}^T \boldsymbol{z} + \left(\boldsymbol{C}_{1,[2]}^T \boldsymbol{z}\right) * \left(\boldsymbol{C}_{2,[2]}^T \boldsymbol{z}\right) + \ldots + \underbrace{\left(\boldsymbol{C}_{1,[N]}^T \boldsymbol{z}\right) * \ldots * \left(\boldsymbol{C}_{N,[N]}^T \boldsymbol{z}\right)}_{\text{N Hadamard products}} , \tag{2}$$

where $\{\boldsymbol{C}_{i,[j]} \in \mathbb{R}^{d \times o}\}_{i,j=1}^{N}$ are learnable parameters. The index i in the matrices $\boldsymbol{C}_{i,[j]}$ corresponds to the i^{th} term in the j^{th} order interactions. This expression reduces the learnable parameters significantly from exponential to $\Theta(N^2 \cdot d \cdot o)$.

We can reduce further the parameters by assuming that certain factors are shared across different layers. The sharing can be achieved by jointly factorizing all the tensors $\{\boldsymbol{\mathcal{W}}^{[n]}\}_{n=1}^{N}$. In Π-Net [75], various such concrete coupled tensor decompositions are utilized to jointly factorize the tensors. A simple recursive expression of that factorization is exhibited below:

$$\boldsymbol{y}_n = \left(\boldsymbol{A}_{[n]}^T \boldsymbol{z}\right) * \left(\boldsymbol{S}_{[n]}^T \boldsymbol{y}_{n-1} + \boldsymbol{b}_{[n]}\right) \,, \tag{NCP}$$

for $n = 2, \ldots, N$ with $\boldsymbol{y}_1 = \left(\boldsymbol{A}_{[1]}^T \boldsymbol{z}\right) * \boldsymbol{b}_{[1]}$ and $\boldsymbol{y} = \boldsymbol{C}\boldsymbol{y}_N + \boldsymbol{\beta}$. The parameters $\boldsymbol{C} \in \mathbb{R}^{o \times k}, \boldsymbol{A}_{[n]} \in \mathbb{R}^{d \times k}, \boldsymbol{S}_{[n]} \in \mathbb{R}^{k \times k}$, $\boldsymbol{b}_{[n]} \in \mathbb{R}^k$ for $n = 1, \ldots, N$, are learnable with $k \in \mathbb{N}$ the rank of the decomposition. That is, the final output $\boldsymbol{y}$ is an N^{th} degree polynomial expansion of the input $\boldsymbol{z}$ and is obtained as an affine transformation of the last recursive term $\boldsymbol{y}_N$. The recursive terms $\{\boldsymbol{y}_n\}_{n=1}^{N}$ provide the intermediate representations. Eq. (NCP) includes few core operations that exist in all major deep learning frameworks and as such, it can be easily implemented. An indicative implementation of Eq. (NCP) is added in Section S.2.2. Note that the learnable parameters are of the order $\Theta(N \cdot d \cdot o)$ owing to the parameters sharing. The recursive formula is obtained by assuming a hierarchical CP decomposition with shared factors as illustrated in [75], while different recursive formulations can be obtained if we change the assumptions or the factorization of the terms.

In practice, there are two relaxations over the formulations above: Firstly, often activation functions are used in between the different terms for stabilizing the training [75]. Secondly, the aforementioned polynomial expansions are often used as polynomial layers, which are then composed sequentially to obtain the final network. For instance, the second polynomial layer of Eq. (NCP) accepts the output of the first polynomial layer of Eq. (NCP) as input. That is, the final network is a composition of the polynomial layers. The total degree of expansion is then the product of the degrees of each polynomial layer. The benefit of such a composition is that it can increase the total degree of expansion without increasing the number of layers significantly. For instance, a product of N polynomial layers of degree 2 results in a 2^N total degree.

Let us now demonstrate two additional applications of the aforementioned polynomial expansion of Eq. (NCP). Fathony et al. [76] apply an elementwise function g on the output of the term $\left(\boldsymbol{A}_{[n]}^T \boldsymbol{z}\right)$ of Eq. (NCP). They consider two special forms of functions g, i.e., the sinusoid and the Gabor filter. In both cases, they illustrate that the final output is a linear combination of Fourier or Gabor bases respectively. Follow-up works further extend the types of filters and add constraints to those [77], [78]. The idea is to use such networks for approximating low-dimensional functions, e.g., given an (x, y) coordinate of an input image output the RGB values of that coordinate. This implicit representation task has been frequently met across a range of domains, such as computer vision, the last few years.

The second application concerns the influential work of StyleGAN [2]. In StyleGAN, a Hadamard-based generator is introduced for the first time in the Generative Adverarial Nets (GANs) [79]. The Hadamard product is performed in the ADAIN layer [80], while the generator structure resembles Eq. (NCP). The authors illustrate how this structure leads to an unsupervised separation of high-level attributes. Follow-up works [81], [82] have further focused on the disentanglement of the high-level attributes using the inductive bias of the Hadamard product.

Besides the vector-based analysis above, a tailored analysis could be performed depending on the type of data/task. The seminal squeeze-and-Excitation block (SENet) [1] relies on an image structure. The output captures pairwise channel correlations of the input $\boldsymbol{Z} \in \mathbb{R}^{hw \times c}$, where h denotes the height, w the width and c the channels of the image. Notation-wise, the output $\boldsymbol{Y}$ is expressed as:

$$\boldsymbol{Y} = (\boldsymbol{Z}\boldsymbol{C}_{[1]}) * r(p(\boldsymbol{Z}\boldsymbol{C}_{[1]})\boldsymbol{C}_{[2]}) \ . \tag{3}$$

The functions p, r represent the global pooling and the replication of the spatial dimensions respectively. The matrices $\boldsymbol{C}_{[1]} \in \mathbb{R}^{c \times \omega}, \boldsymbol{C}_{[2]} \in \mathbb{R}^{\omega \times c}$ are learnable. The output of the global pooling $p(\boldsymbol{Z}\boldsymbol{C}_{[1]})$ is a vector of size ω. Follow-up works have extended the pairwise channel correlations [83], [84], [85]. Indicatively, the selective kernel nets [86] perform a minor re-parametrization of Eq. (3) to enable different receptive fields on the features. Beyond recognition, SENet has been applied to image compression [87], image super-resolution [88] and denoising [89], semantic segmentation [90], medical image segmentation [91], [92], [93] and point cloud completion [94]. Follow-up works have expanded on the role of channel and spatial correlations, e.g., in the context of semantic image synthesis [95] or medical image segmentation [96], [97].

High-order interactions can be beneficial beyond the representations of data that we focused above. For instance, high-order interactions are used for expressing non-linear positional encodings [98], [99]. Positional encodings express the absolute or relative position of an element in a sequence, when the ordering is important. In [100], they argue that capturing the relative positioning is more complex in 3D point clouds, since points are unevenly distributed in the space.

Overall, networks capturing higher-order interactions have exhibited state-of-the-art results in 3D modeling [101], [102], pose estimation [103], image generation [2], [75], audio generation [104], image recognition [1], [65], segmentation [105] and multimodal representation learning [106], [107], [108], [109]. We hypothesize that this can be partly attributed to the increased expressivity of such networks, as detailed in Section 6.

TABLE 2: *Important* works in the category of multimodal fusion.

Task	Method	Publication
Visual Question Answering (VQA)	[11]	ICCV'15
VQA	MRN [121]	NeurIPS'16
VQA	MUTAN [122]	ICCV'17
Semantic image generation	SPADE [3]	CVPR'19
Knowledge graph completion	RotatE [123]	ICLR'19
Text-guided image retrieval	VAL [124]	CVPR'20
Video-Text Retrieval	HGR [125]	CVPR'20
Vision-and-Language Navigation	[126]	NeurIPS'20
Conditional generation	CoPE [38]	NeurIPS'21

3 Multimodal Fusion

In nature, phenomena can be described using data from various sensors or detectors. For example, humans observe objects and scenes while associating sounds or odors with them. The data distribution from a different sensor is called *modality* [110]. This section focuses on multimodal fusion, which involves tasks that use data from multiple sensors, such as visual and auditory data, and their interactions. The concept of data fusion dates back to at least the seminal work of Hotelling [111]. The joint analysis of multiple datasets has been explored since [112], [113], with significant advancements in the 1970s through factor analysis [114], [115]. Since then, these ideas have expanded beyond psychometrics and chemometrics.

In machine learning, the earliest reference to a multimodal system is the combination of speech and video (image) processing by [116]. Early studies showed that the interaction between auditory and visual information is non-linear rather than purely additive [117], [118]. Motivated by this complexity, Yuhas et al. [119] employed neural networks for multimodal tasks, suggesting that similar frameworks could be beneficial in other multimodal contexts. In recent years, the rapid development of electronic devices and social networks has enabled the unprecedented collection of multimedia sources, often as multimodal data. The Hadamard product has proven particularly useful in multimodal fusion, as demonstrated in visual question answering (VQA) challenges [120]. Suppose we have two input variables, $\boldsymbol{z}_{\text{I}}$ and $\boldsymbol{z}_{\text{II}} \in \mathbb{R}^d$, from different modalities. We aim to fuse the information captured across these modalities with many important works, e.g., as mentioned in Table 2, leveraging Hadamard product for the task.

The straightforward idea is to fuse the data from two streams using a function of the form $f_1(\boldsymbol{z}_{\text{I}}) * f_2(\boldsymbol{z}_{\text{II}})$ for appropriate functions f_1, f_2. In [127], the image and word embeddings are merged in the GAN discriminator using this fusion. In the visual question answering (VQA) of [11], [122], [128], [129], [130], [131], [132], [133], [134] or in video question answering [135], a similar formula is used to fuse the visual and the text embeddings. The same formula with a single Hadamard product emerges in tasks where more than one variables are available [136], [137], [138], [139],

[140], [141], [142]. Lastly, in the hashtag prediction task, the single Hadamard product is preferred over alternative fusion techniques [143].

The aforementioned works perform feature fusion using a single Hadamard product per modality. However, utilizing ideas from Section 2, we can capture high-order cross-interactions across modalities. That is, we can capture both the auto- and the cross-correlations of the input variables $\boldsymbol{z}_{\text{I}}, \boldsymbol{z}_{\text{II}}$. A recursive formulation that can capture such higher-order interactions is the following [38]:

$$\boldsymbol{y}_n = \left(\boldsymbol{A}_{[n,I]}^T \boldsymbol{z}_{\text{I}} + \boldsymbol{A}_{[n,II]}^T \boldsymbol{z}_{\text{II}}\right) * \left(\boldsymbol{S}_{[n]}^T \boldsymbol{y}_{n-1} + \boldsymbol{b}_{[n]}\right) \ , \quad (4)$$

for $n = 2, \ldots, N$ with $\boldsymbol{y}_1 = \left(\boldsymbol{A}_{[1,I]}^T \boldsymbol{z}_{\text{I}} + \boldsymbol{A}_{[1,II]}^T \boldsymbol{z}_{\text{II}}\right) * \boldsymbol{b}_{[1]}$. The parameters $\boldsymbol{C} \in \mathbb{R}^{o \times k}, \boldsymbol{A}_{[n,\phi]} \in \mathbb{R}^{d \times k}, \boldsymbol{S}_{[n]} \in \mathbb{R}^{k \times k}, \boldsymbol{b}_{[n]} \in \mathbb{R}^k$ for $\phi = \{I, II\}$ are learnable. Then, the output is $\boldsymbol{y} = \boldsymbol{C}\boldsymbol{y}_N + \boldsymbol{\beta}$. That is, the output is an N^{th} degree polynomial expansion with respect to the input variables $\boldsymbol{z}_{\text{I}}, \boldsymbol{z}_{\text{II}}$ and is expressed as the affine transformation of the last recursive term $\boldsymbol{y}_N$. This formulation can be thought of as an extension of Eq. (NCP) for the case of two input variables. Having separate matrices $\boldsymbol{A}_{[n,I]}, \boldsymbol{A}_{[n,II]}$ associated with each respective input variable is beneficial when the input variables differ substantially in the information they represent, e.g., low-resolution image versus a data-class label. Special forms[1] of Eq. (4) have been considered in the literature [21], [22], [23], [121], [144], [145], [146], [147], [148], [149], [150], [151], [152], [153], [154], [155], [156], [157], [158].

In particular, the formulation is often applied in the case of text-to-image generation [159], [160], where one variable captures the text information and another one the input noise, which is sampled from the Gaussian distribution. Similarly, in the text-guided image manipulation of [161], the formulation of Eq. (4) is applied with $\{\boldsymbol{A}_{[n,II]} = \boldsymbol{0}\}_{n=2}^N$, where $\boldsymbol{z}_{\text{II}}$ denotes the text information and $\boldsymbol{z}_{\text{I}}$ captures the visual information. That is, the text embeddings are explicitly utilized only on the first Hadamard product, while the visual embeddings are used in subsequent Hadamard products. In [162], [163], the opposite idea was illustrated. That is, $\{\boldsymbol{A}_{[n,I]} = \boldsymbol{0}\}_{n=2}^N$, where $\boldsymbol{z}_{\text{I}}$ captures the visual information. In [164], they synthesize an image given the bounding boxes of the objects (layout) as conditional information. If we denote the input noise, which is sampled from the Gaussian distribution, as $\boldsymbol{z}_{\text{I}}$ and the conditional information (which is the layout) as $\boldsymbol{z}_{\text{II}}$, [164] considers $\{\boldsymbol{A}_{[n,I]} = \boldsymbol{0}\}_{n=2}^N$. A similar idea is used in [165] for VQA, where $\boldsymbol{z}_{\text{I}}$ captures the visual information, $\boldsymbol{z}_{\text{II}}$ the text information and $\{\boldsymbol{A}_{[n,I]} = \boldsymbol{0}\}_{n=2}^N$.

The seminal work of SPADE [3] can be cast as a form of Eq. (4) with respect to a single variable similarly to [165]. This variable corresponds to the semantic layout and is the conditional input in SPADE. If $\boldsymbol{z}_{\text{II}}$ is the conditional input, SPADE assumes that $\boldsymbol{A}_{[n,I]} = \boldsymbol{0}$ for $n = 2, \ldots, N$ in Eq. (4). Object-shape information [166] have been added to SPADE to extract additional features from the conditional variable (i.e., the semantic map). An interesting extension is to include additional variables, e.g., to express style information [167].

1. Even though activation functions are often used among the terms of Eq. (4) in various works, those do not affect the Hadamard product that is the main focus here. Therefore, we will omit such element-wise activation functions to avoid cluttering the notation.

The recursive form of Eq. (4) can be extended to facilitate an arbitrary number of variables [38]. For instance, for a third input variable $\boldsymbol{z}_{\text{III}}$, we apply an affine transformation with learnable parameters $\{\boldsymbol{A}_{[n,III]}\}_{n=1}^N$ and then augment the first term of Eq. (4) with $\boldsymbol{A}_{[n,III]}^T \boldsymbol{z}_{\text{III}}$. Such a formulation arises in text-guided image generation [168], where they capture correlations between the conditional information, i.e., text, the intermediate representations, and the input noise. In the VQA framework of [169], they capture the correlations between the image, question and answer representations using a formula of the form $(\boldsymbol{A}_{[I]}\boldsymbol{z}_{\text{I}}) * (\boldsymbol{A}_{[II]}\boldsymbol{z}_{\text{II}}) * (\boldsymbol{A}_{[III]}\boldsymbol{z}_{\text{III}})$ with $\boldsymbol{A}_{[I]}, \boldsymbol{A}_{[II]}, \boldsymbol{A}_{[III]}$ learnable. This ternary relationship is further extended in [170] for capturing ternary image-text retrieval (TITR). To be precise, they evaluate TITR in recipe retrieval (where the ingredient text, the instruction text and the food image are the three variables) and in fashion search (where the original image, the modified image and the text are the three variables). The same expression is captured in [171] between text embeddings, visual embeddings and sentimental attributes for sentimental VQA. Note that the aforementioned formula is a special case of Eq. (4).

Alternative formulations have been also considered in the literature. For instance, a property of the Hadamard product connecting it with the Fast Fourier Transform (FFT) is exhibited in [12] for VQA. That is, the convolution of two vectors $\boldsymbol{z}_{\text{I}}, \boldsymbol{z}_{\text{II}}$ (i.e., word and visual embeddings) can be computed as $\text{FFT}^{-1}(\text{FFT}(\boldsymbol{z}_{\text{I}}) * \text{FFT}(\boldsymbol{z}_{\text{II}}))$. Then, they use the Hadamard product twice to fuse the two modalities. In [98], multiple Hadamard products are considered for fusing the visual and the text information for VQA. The co-attention of [172] includes two pathways: one that captures high-order correlations of the visual information $\boldsymbol{z}_{\text{I}}$ and one that captures the respective correlations for text $\boldsymbol{z}_{\text{II}}$. Concretely, in the first pathway they use $\{\boldsymbol{A}_{[(1),n,II]} = \boldsymbol{0}\}_{n=2}^N$, while in the second $\{\boldsymbol{A}_{[(2),n,I]} = \boldsymbol{0}\}_{n=2}^N$, where we added the $(1), (2)$ indices to separate the pathways.

Another line of research involves modifying the popular attention mechanism [10] for multimodal tasks. For instance, [173] modifies the self-attention block using Hadamard product, and applies this for image captioning. A similar modification is used in the attention mechanism of [174] for image captioning and VQA. Both matrix multiplications between the (transformed) inputs of the attention are converted to Hadamard products in [175]. Attention has also been modified to include a multimodal fusion of visual and text embeddings with Hadamard product in the query and key attributes in [176].

Frequently, visual/text grounding tasks require reasoning about the relationship between objects. For instance, in visual grounding (VG) that idea is to predict the location of the most relevant object given a query expressed in natural language. Message-passing with Hadamard product [125], [177] can be used for such relational reasoning. In [178], they construct a message passing graph network, where each object is represented by a node. They use the Hadamard product both for updating the current node features (which they mention as the context features) and merging the visual and the text features. The framework is augmented in [179] to improve the consistency between different text queries. In [180], they rely on message passing with Hadamard

product to capture the visual and text interactions. A similar idea with message passing has been used for agent navigation [126]. The agent needs to follow instructions, while it has access to the visual scene. The network of the agent uses the Hadamard product to deliver relevant information from the text instructions to the scene. In [181], message passing with the Hadamard product is utilized to capture interactions between the visual and the text embeddings for localizing actions in a video stream.

The graph structure is also frequently assumed in knowledge graphs beyond the aforementioned message passing. Knowledge graphs consist of a number of triplets $(\boldsymbol{h}, \boldsymbol{r}, \boldsymbol{t})$ that relate a head entity $\boldsymbol{h}$ and a tail entity $\boldsymbol{t}$ with a relationship $\boldsymbol{r}$. Such a triplet can be the following: EPFL (head entity) is located in Switzerland (tail entity). Often, there are missing information in a knowledge graph, e.g., not all the relationships are provided, and we need to perform knowledge graph completion. The Hadamard product holds a key role for the knowledge graph completion of RotatE [123]. The entities and relationships are mapped to complex space vectors, with $\boldsymbol{r}$ also being a unitary vector (i.e., $|r_i| = 1$ with $\boldsymbol{r} = [r_1, r_2, \ldots, r_d]$). Then, the relationship in RotatE is modeled as a rotation from the head to the tail entity as $\boldsymbol{t} = \boldsymbol{h} * \boldsymbol{r}$. In [182], they augment RotatE by including convolutional correlations with an additional Hadamard product. The idea of RotatE is extended to include semantic hierarchies in HAKE [183]. RotatE has also been modified to capture correlations among different triplets [184]. The work of [185] uses the Hadamard product to implement a similar idea with quaternions. SimplE [186] requires the Hadamard product to capture correlations between entities from other triplets, while CrossE [187] learns "interacting" embeddings that jointly capture the entity-relationship correlations.

Beyond the aforementioned applications, the Hadamard product has been used in medical imaging and neuroimaging, where different modalities are often required for a diagnosis [188]. For instance, the fusion of magnetic resonance imaging (MRI) and positron emission tomography (PET) modalities [40], or fusion of MRI and ultrasound images [189] have utilized the Hadamard product. We expect that the adoption of deep learning approaches in neuroimaging can lead to further applications of the Hadamard product [190]. The Hadamard product is used for fusing visual and text embeddings in the medical visual question answering (Med-VQA) [191] of [192], [193].

Another application of the Hadamard product is in remote sensing, where information about the earth are collected from a distance, e.g., airborne or satellite images. Even though monitoring the surface of the earth is important, deploying and maintaining sensors is expensive and trade-offs (e.g., images of low spatial resolution) are often made. Multimodal fusion can then be used to extract accurate information about complex scenes, e.g., by merging information from different wavelengths/sensors [54]. In [194], a single Hadamard product is used to guide the image representations using class-information, while multiple Hadamard products are used in [195]. The attention mechanism is modified to include Hadamard product in [196], [197], in a similar way as mentioned above for image captioning. In [198], they argue that RGB information can provide useful features, but it also results ambiguity because of the complex texture involved, thus auxiliary information are required. They propose a block in the form of Eq. (4) (for $N = 2$) for the fusion of those modalities.

TABLE 3: *Important* works in the category of adaptive modulation.

Task	Method	Publication
Sequence learning	LSTM [8]	Neural computation'97
Image recognition	DropConnect [199]	ICML'13
Vision, speech, NLP	Dropout [200]	JMLR'14
Sequence learning	GRU [9]	SSST'14
Image inpainting	[4]	CVPR'16
Image inpainting	[5]	CVPR'18
Language modeling	T-Few [201]	NeurIPS'22

4 Adaptive Modulation

Int this section, we focus on the role of the Hadamard product in masking. Frequently, only part of the input should be considered, e.g., in the case of image inpainting we want the occluded part to be filled in using the rest of the input image. Similarly, in image generation we might want to synthesize the same object with different background, or equivalently synthesize the same (human) voice with different auditory backgrounds. We examine below cases where masking is utilized, either in the form of a binary mask or in a soft mask or even as a dynamic modulation of the weights. Concretely, in Section 4.1, we review various masking and adaptive modulation techniques, while on Section 4.2 we attend to the recurrent models that perform a dynamic modulation of the inputs. Few important works on the topic are mentioned in Table 3.

4.1 Masking with Hadamard product

Binary masks: In inpainting, a binary mask is often provided as input along with the image to be filled in [4], [5]. The binary mask indicates which values should be filled in (marked with values 1), while the rest of the mask attains the value 0. Then, a Hadamard product is utilized either in the forward pass [202], [203], [204], [205] or in the backward pass [5]. If the Hadamard product is used in the forward pass, it expresses effectively a branching function for each pixel. That is, if the pixel should remain, it maintains the same value, otherwise a function is applied to that pixel. Beyond image inpainting, binary masks have been used in video inpainting [206], depth completion [207], image blending [208], 3D mesh generation [209], image editing [210]. In [211], binary semantic class masks are used for semantic-guided scene generation. A binary tree mask is constructed in [212] to restrict the adjacency matrix of the word graph for text-guided image segmentation.

Beyond the fixed, binary mask, we can consider updating the binary mask, e.g., by partially reducing the masked area [213], [214]. Pruning the weights of a neural network is an exemplary usecase of binary masks. Mask values of 0 effectively prune the corresponding weights, while values of 1 retain the original weights [25], [215], [216]. The seminal work of Dropout [217] belongs in this category. Dropout aims at reducing overfitting, by masking at random few elements in the intermediate representations. If $\boldsymbol{y}_n$ denotes the intermediate representations at the n^{th} layer

of a network (cf. Eq. (S2)) and $\boldsymbol{\gamma} \sim \mathcal{B}$ denotes sampling from the Bernoulli distribution $\mathcal{B}$, then Dropout is expressed as:

$$\boldsymbol{x}_{n+1} = \boldsymbol{\gamma} * \boldsymbol{y}_n \ , \tag{5}$$

where the $\boldsymbol{x}_{n+1}$ is the input for the next layer, i.e., $\boldsymbol{y}_{n+1} = \sigma(\boldsymbol{S}_n \boldsymbol{x}_{n+1})$ for appropriate $\sigma, \boldsymbol{S}_n$. Numerous extensions have been introduced to Dropout, such as sampling from a Gaussian distribution [200], adapting the dropout probability [218], or regularizing dropout for representation consistency [219]. A principled understanding of Dropout has sparked the interest of the community [220]. A more thorough overview of the developments on Dropout can be found in related surveys [51]. An idea similar to Eq. (5) was proposed as a feature-wise transformation layer for regularizing the representations [221] or the channels of the representations [222].

Hadamard product is also used for causal language modeling with linear attention. Attention and self-attention mechanisms [10] have a dramatic influence across different domains the last few years [7], [49], [223], [224]. The attention mechanism, which was originally introduced for machine transduction, reduced the computational complexity over the previously used recurrent models for such sequence-to-sequence tasks. Given a sequence of length n and the input matrices $\boldsymbol{Q} \in \mathbb{R}^{n \times d_k}, \boldsymbol{K} \in \mathbb{R}^{n \times d_k}$ and $\boldsymbol{V} \in \mathbb{R}^{n \times d_v}$, the attention mechanism is expressed as follows:

$$\boldsymbol{Y}_{SA} = \sigma_s \left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}} \right) \boldsymbol{V} \ , \tag{6}$$

where σ_s is frequently the row-rise softmax activation function. The self-attention (SA) used in practice in [10] relies on a single sequence, i.e., the $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ are affine transformations of the common input $\boldsymbol{Z}$, e.g., a sequence of word embeddings. That is, the input $\boldsymbol{Z} \in \mathbb{R}^{n \times d}$ is transformed into $\boldsymbol{Q} = \boldsymbol{Z}\boldsymbol{W}_q, \boldsymbol{K} = \boldsymbol{Z}\boldsymbol{W}_k$ and $\boldsymbol{V} = \boldsymbol{Z}\boldsymbol{W}_v$ with learnable parameters $\{\boldsymbol{W}_\tau \in \mathbb{R}^{d \times d_k}\}_{\tau \in \{q,k\}}$ and $\boldsymbol{W}_v \in \mathbb{R}^{d \times d_v}$.

To address the quadratic complexity of self-attention, linear self-attention was proposed [41], where σ_s is replaced by an identity matrix. This approach has been adopted in several recent advancements in language models [225], [226], [227]. During language-model training, the task is to predict the next token based on all previous tokens. That would effectively mean that to output a single sentence consisting of m tokens, we would need to make m forward passes, which is very costly. Instead, a masking mechanism can enable to make the prediction in a single forward pass ensuring that a query cannot access keys from future tokens. Mathematically, this process can be formulated as follows [2]:

$$\boldsymbol{Y}_{\text{LIN_SA}} = \left\{ \left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}} \right) * \boldsymbol{R} \right\} \boldsymbol{V} \ , \tag{7}$$

where $\boldsymbol{R}$ is the masking matrix, i.e., a lower triangular matrix with entries 1, as present in Fig. 1(c). To provide further information on masked language modeling, we include an indicative PyTorch implementation in Section S.2.1.

Binary masks, in which the mask is predicted as the output of a network or an optimization process, have naturally emerged in image/video synthesis tasks. The method of [228] continuously updates the distribution of masks, since they train a mask generator for explicitly disentangling foreground from background in image synthesis. Similarly, in [229], they feed two consecutive frames (of a video) into a network to estimate the foreground/background mask and then condition the synthesis of the new frame into those masks. Another application is outlier detection, where a learnable binary mask is used to filter examples that do not belong to the training distribution [230].

Soft masks: Beyond the aforementioned applications of binary masks, we could consider masks as probabilities, i.e., the probability a pixel is correct or needs to be refined. Each element in this mask accepts a value in the range $[0, 1]$. This mask is referred to as "soft mask" henceforth.

The image inpainting of [231] uses soft masks to update the regions that have been completed and which need to be refined. In the video captioning of [147], the region proposals are converted to a soft mask and then fused with the visual embeddings for determining which regions are relevant for video captioning. In [232], the Hadamard product is used to emphasize the anomalous regions in medical images. In [95], an edge-based mask is created to guide the semantic image generation. In the spiking neural networks of [233], a self-feedback mechanism is inserted using the Hadamard product. Various perspectives on soft masks with the utilization of the Hadamard product have also been considered for tumor detection and classification [234], [235], text-guided image manipulation [163], [236], image inpainting [237], semantic image synthesis [238], deraining [239], cancer diagnosis [240], medical imaging super-resolution [241], recommender systems [242] and click-through rate prediction [243]. In the approaches of [244], [245], [246], the scaling matrix is only constrained to contain positive values, but not explicitly constrained in the $[0, 1]$ interval.

Lastly, we consider the case of adaptive modulation of the input, where typically there are two representations and we consider the weighted average of those element-wise. For inputs $\boldsymbol{A}$ and $\boldsymbol{B}$, this is expressed as:

$$\boldsymbol{Y} = \boldsymbol{R} * \boldsymbol{A} + (\boldsymbol{1} - \boldsymbol{R}) * \boldsymbol{B} \, , \tag{8}$$

where the $\boldsymbol{R}$ is the weighting matrix that accepts values in the $[0, 1]$ interval. Eq. (8) is also frequently used in multimodal fusion (complementary to the aforementioned techniques in Section 3). In the image manipulation of [247], the textual instructions are computing the weighting matrix, and the visual and text embeddings are fused using the Eq. (8). In [248], this equation is used for multimodal medical image fusion, where there are two source images and the weighting matrix is obtained as the output of the decoder. In the face swapping of [249], the attribute embeddings and the identity embeddings are fused using a weighting matrix that is computed from the visual embeddings. Eq. (8) has been used for fine-grained image generation of the foreground/background [250]. The work of [251] relies on the previous fine-grained generation to perform disentangled image generation with weak supervision. The RelTransformer [252] captures the relationships between objects (entities) using Eq. (8). Masking through the Hadamard product is used for reducing the unimodal biases in VQA [253]. Concretely, the mask is used to alter

2. The nonlinear feature mapping for $\boldsymbol{Q}$ and $\boldsymbol{K}$ are omitted for notation simplification.

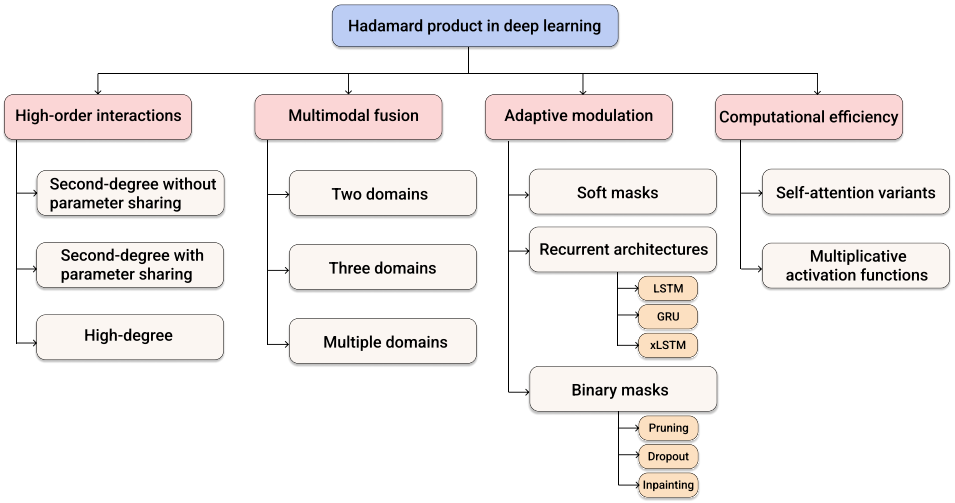


Fig. S2: Taxonomy of Hadamard product in deep learning. The category of high-order interactions is often divided by the degree of interactions, with a more fine-grained taxonomy being whether there is parameter-sharing, i.e., Eq. (2) vs Eq. (NCP). Similarly, in multimodal fusion, the number of domains is a fundamental separation, with techniques such as VQA belonging to two domains. We believe in the following years, works will increasingly focus on multiple domains for general applications, which simulates how humans perceive and process multiple domains.

term, i.e., $\boldsymbol{y} = \boldsymbol{C}\boldsymbol{y}_N + \boldsymbol{\beta}$. The parameters $\boldsymbol{\beta} \in \mathbb{R}^o, \boldsymbol{C} \in \mathbb{R}^{o\times k}, \boldsymbol{S}_{[n]} \in \mathbb{R}^{k\times k}$, $\boldsymbol{b}_{[n]} \in \mathbb{R}^k$ for $n = 1, \ldots, N$ are learnable. The symbol σ represents an activation function (e.g., Rectified Linear Unit (ReLU), hyperbolic tangent (tanh), sigmoid), which induces non-linearity to the network. In other words, the intermediate representations $\boldsymbol{y}_n$ can express non-linear functions of the inputs.

In practice, extensions of Eq. (S2) to facilitate components such as normalization schemes [322] or residual connections [323], can be included with a minor re-parametrization. For instance, the influential residual block can be expressed as $\boldsymbol{y}_n = \sigma\left(\boldsymbol{S}_{[n]}^T \boldsymbol{y}_{n-1} + \boldsymbol{b}_{[n]}\right) + \boldsymbol{y}_{n-1}$, where essentially only an identity of the output from the previous layer is added. In the rest of the manuscript, unless mentioned explicitly otherwise σ denotes an element-wise activation function, e.g., sigmoid or ReLU, below. Similarly, in the linear layers $\boldsymbol{W}\boldsymbol{z}+\boldsymbol{\beta}$ with learnable parameters $\boldsymbol{W}, \boldsymbol{\beta}$ and input $\boldsymbol{z}$ the bias term $\boldsymbol{\beta}$ might be omitted for simplicity.

TABLE S4: Indicative list of public implementations (frequently from authors) on many of the works cited in the survey.

Citation	*Authors' implementation*	*Implementation*
[1]	✓	https://github.com/hujie-frank/SENet
[3]	✓	https://github.com/NVlabs/SPADE
[4]	✓	https://github.com/pathak22/context-encoder
[5]	✓	https://github.com/JiahuiYu/generative_inpainting
[6]		https://github.com/anantzoid/Language-Modeling-GatedCNN
[15]		https://github.com/lucidrains/FLASH-pytorch
[18]	✓	https://github.com/Allencheng97/Multilinear_Operator_Networks
[20]	✓	https://github.com/qhfan/RMT
[22]	✓	https://github.com/omerbt/MultiDiffusion
[27]	✓	https://github.com/JulianGuerreiro/LayoutFlow
[31]	✓	https://github.com/OpenNLPLab/HGRN2
[34]	✓	https://github.com/ma-xu/Rewrite-the-Stars
[38]	✓	https://github.com/grigorisg9gr/polynomial_nets_for_conditional_generation
[42]	✓	https://github.com/LIONS-EPFL/pntk
[67]		https://github.com/lucidrains/g-mlp-pytorch
[72]	✓	https://github.com/grigorisg9gr/regularized_polynomials
[74]	✓	https://github.com/grigorisg9gr/polynomials-for-augmenting-NNs
[76]	✓	https://github.com/boschresearch/multiplicative-filter-networks
[84]	✓	https://github.com/Jongchan/attention-module
[85]	✓	https://github.com/cfzd/FcaNet
[86]	✓	https://github.com/implus/SKNet
[88]	✓	https://github.com/yulunzhang/RCAN
[89]	✓	https://github.com/saeed-anwar/RIDNet
[94]	✓	https://github.com/paul007pl/VRCNet
[95]	✓	https://github.com/Ha0Tang/ECGAN
[99]	✓	https://github.com/woodfrog/vse_infty
[102]	✓	https://github.com/grigorisg9gr/polynomial_nets
[105]	✓	https://github.com/microsoft/FocalNet
[106]	✓	https://github.com/AmingWu/CCN
[107]	✓	https://github.com/simon-ging/coot-videotext
[121]	✓	https://github.com/jnhwkim/nips-mrn-vqa
[122]	✓	https://github.com/Cadene/vqa.pytorch
[124]	✓	https://github.com/yanbeic/VAL
[126]	✓	https://github.com/YicongHong/Entity-Graph-VLN
[169]	✓	https://github.com/aioz-ai/ICCV19_VQA-CTI
[181]	✓	https://github.com/crodriguezo/DORi
[185]	✓	https://github.com/cheungdaven/QuatE
[186]	✓	https://github.com/Mehran-k/SimplE
[187]	✓	https://github.com/wencolani/CrossE
[192]	✓	https://github.com/Awenbocc/med-vqa
[196]	✓	https://github.com/athauna/AFNet
[200]	✓	https://github.com/nitishsrivastava/deepnet
[205]	✓	https://github.com/GouravWadhwa/Hypergraphs-Image-Inpainting
[208]	✓	https://github.com/lhoyer/improving_segmentation_with_selfsupervised_depth
[209]	✓	https://github.com/dariopavllo/convmesh
[212]	✓	https://github.com/spyflying/LSCM-Refseg
[213]	✓	https://github.com/NVIDIA/partialconv
[225]	✓	https://github.com/microsoft/unilm/tree/master/retnet
[229]	✓	https://github.com/Gabriel-Huang/Layered-Controllable-Video-Generation
[231]		https://github.com/Vious/LBAM_Pytorch
[236]		https://github.com/Psarpei/GanVinci
[242]	✓	https://github.com/allenjack/HGN
[249]		https://github.com/justin4ai/hearnet-pytorch
[253]	✓	https://github.com/cdancette/rubi.bootstrap.pytorch
[256]	✓	https://github.com/x-zho14/ProbMask-official
[258]	✓	https://github.com/arunmallya/piggyback
[269]	✓	https://github.com/jzilly/RecurrentHighwayNetworks
[273]	✓	https://github.com/wuch15/Fastformer
[276]	✓	https://github.com/NVlabs/GatedDeltaNet
[304]	✓	https://github.com/megaelius/PNVerification

S.2 PRACTICAL IMPLEMENTATION

A list of public implementations of works cited in the survey can be found below in Table S4. In addition, below we reference few core implementations using the popular PyTorch. These are mostly for instructional purposes, while we recommend to the interested reader to follow the public implementation of the authors to reproduce the results of the respective paper.

S.2.1 PyTorch example of linear attention for language modeling

We paste below an instructional implementation of language modeling with linear attention.

```python
from torch import nn
import torch

def causal_mask(seq_len):
    """
    Creates a lower-triangular matrix of shape [seq_len, seq_len] with element one for masking
    """
    mask = torch.tril(torch.ones(seq_len, seq_len))
    return mask  # [seq_len, seq_len]

class LinearSelfAttention(nn.Module):
    """
    A simple linear self-attention block with causal masking applied by Hadamard product.
    """
    def __init__(self, hidden_dim):
        super(LinearSelfAttention, self).__init__()
        self.W_q = nn.Linear(hidden_dim, hidden_dim)
        self.W_k = nn.Linear(hidden_dim, hidden_dim)
        self.W_v = nn.Linear(hidden_dim, hidden_dim)

    def forward(self, z):
        """
        Input z: a batch of sequences with embedding [batch_size, seq_len, hidden_dim]
        Returns:
        output: [batch_size, seq_len, hidden_dim]
        """
        batch_size, seq_len, _ = z.size()

        # Compute queries, keys, values
        Q = self.W_q(z)  # [batch_size, seq_len, hidden_dim]
        K = self.W_k(z)  # [batch_size, seq_len, hidden_dim]
        V = self.W_v(z)  # [batch_size, seq_len, hidden_dim]

        # Compute attention scores
        scores = torch.bmm(Q, K.transpose(1, 2))  # [batch_size, seq_len, seq_len]

        mask = causal_mask(seq_len)  # [seq_len, seq_len]
        mask = mask.unsqueeze(0).expand(batch_size, -1, -1)  # [batch_size, seq_len, seq_len]
        scores = scores * mask  # masking with hadamard product for future positions

        # Multiply scores by V to get the output
        output = torch.bmm(scores, V)  # [batch_size, seq_len, hidden_dim]

        return output

class LanguageModel(nn.Module):
    """
    Simple language model for next token prediction, with an embedding layer,
    linear self-attention layer, and output projection.
    """
    def __init__(self, vocab_size, hidden_dim):
        super(LanguageModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.linear_self_attn = LinearSelfAttention(hidden_dim)
        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, z):
        """
        Input z: a batch of sequence with token index
        [batch_size, seq_len]
        Returns:
        logits: [batch_size, seq_len, vocab_size]
        """
        # Get token embedding
        emb = self.embedding(z)  # [batch_size, seq_len, hidden_dim]

        # Apply masked linear self-attention
        attn_output = self.linear_self_attn(emb)  # [batch_size, seq_len, hidden_dim]

        # Output projection to vocabulary size
        logits = self.fc_out(attn_output)  # [batch_size, seq_len, vocab_size]

        return logits
```